LECTURE 01

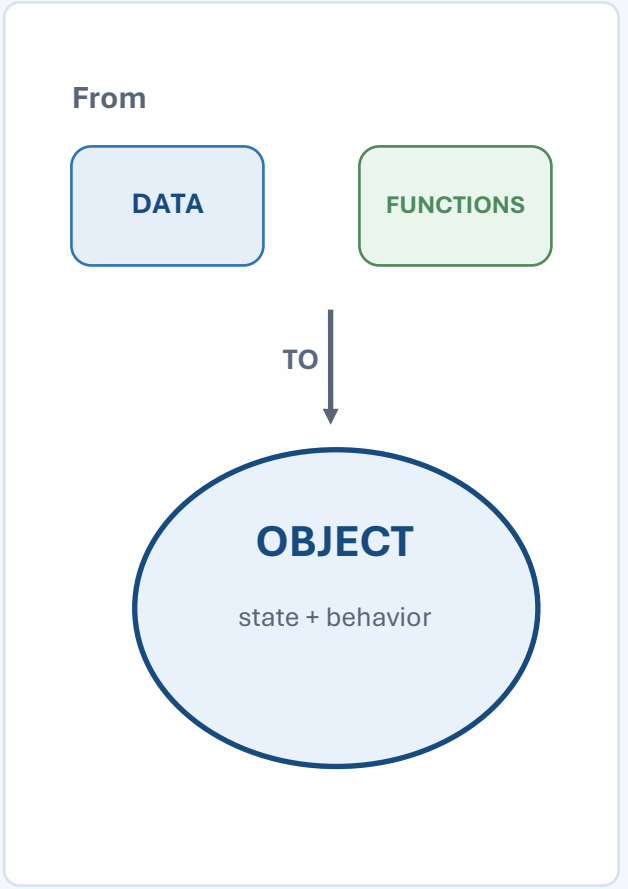
Programming Paradigms & Motivation for OOP

CSC241 — Object-Oriented Programming

Fall 2026 • COMSATS University Islamabad, Lahore Campus

Dr. Muhammad Shahid Bhatti • Assistant Professor
Department of Computer Science • Office H-17

GitHub course repository: github.com/mshahidbhatti/CSC241-OOP

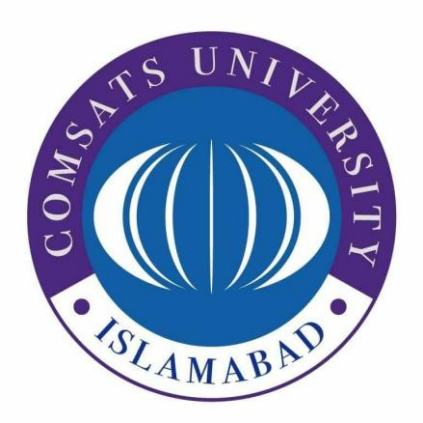


Reference: Lecture plan • Liang, Ch. 1

Instructor & course resources

COURSE ACCESS

Where to find lecture material, examples, and course code.



Dr. Muhammad Shahid Bhatti

Assistant Professor

Department of Computer Science

Office H-17

CSC241 — Object-Oriented Programming

Fall 2026

COMSATS University Islamabad

Lahore Campus

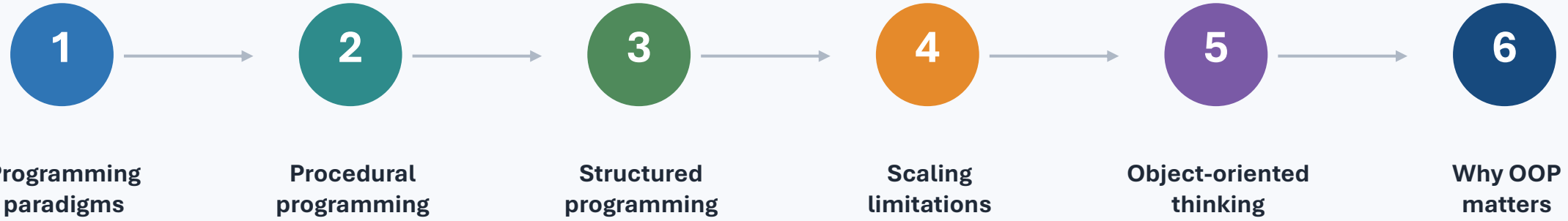
GitHub repository: https://github.com/mshahidbhatti/CSC241-OOP

Repository purpose: lecture slides • live-coding examples • labs/exercises • course resources. The repository is private; access requires authorization.

Lecture roadmap

ROADMAP

We will move from “how programs execute” to “how software is organized.”



Why did software engineers need a different way to structure large programs?

Learning outcomes

OUTCOMES

By the end of this lecture, you should be able to...

- Distinguish procedural, structured, and object-oriented programming at a conceptual level.
- Explain why procedural programs become difficult to evolve as systems grow.
- Describe the central OOP idea: objects combine state with the behavior that operates on that state.
- Recognize situations where OOP improves modularity, maintenance, and collaboration.
- Read a small Java example and identify data, operations, and candidate objects.

Today is about the WHY of OOP. Detailed class syntax begins in later lectures.

What is a programming paradigm?

FOUNDATION

A paradigm is a broad style for organizing computation and reasoning about programs.

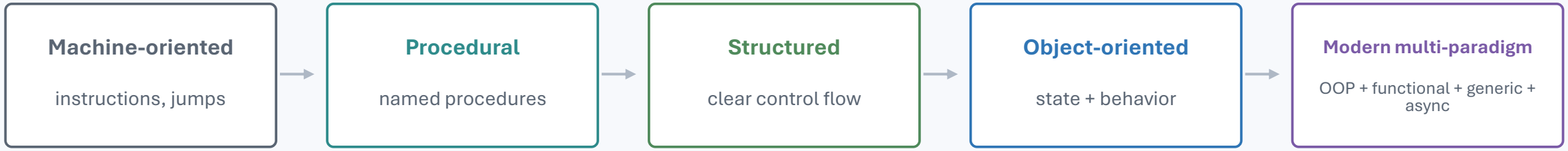
Procedural	Object-Oriented	Functional
Organize around procedures/functions	Organize around objects	Organize around functions & expressions

Most modern languages are multi-paradigm. The important skill is choosing a structure that matches the problem.

A simplified evolution of programming styles

HISTORY

Each step addressed growing complexity—not because the previous style became “wrong.”



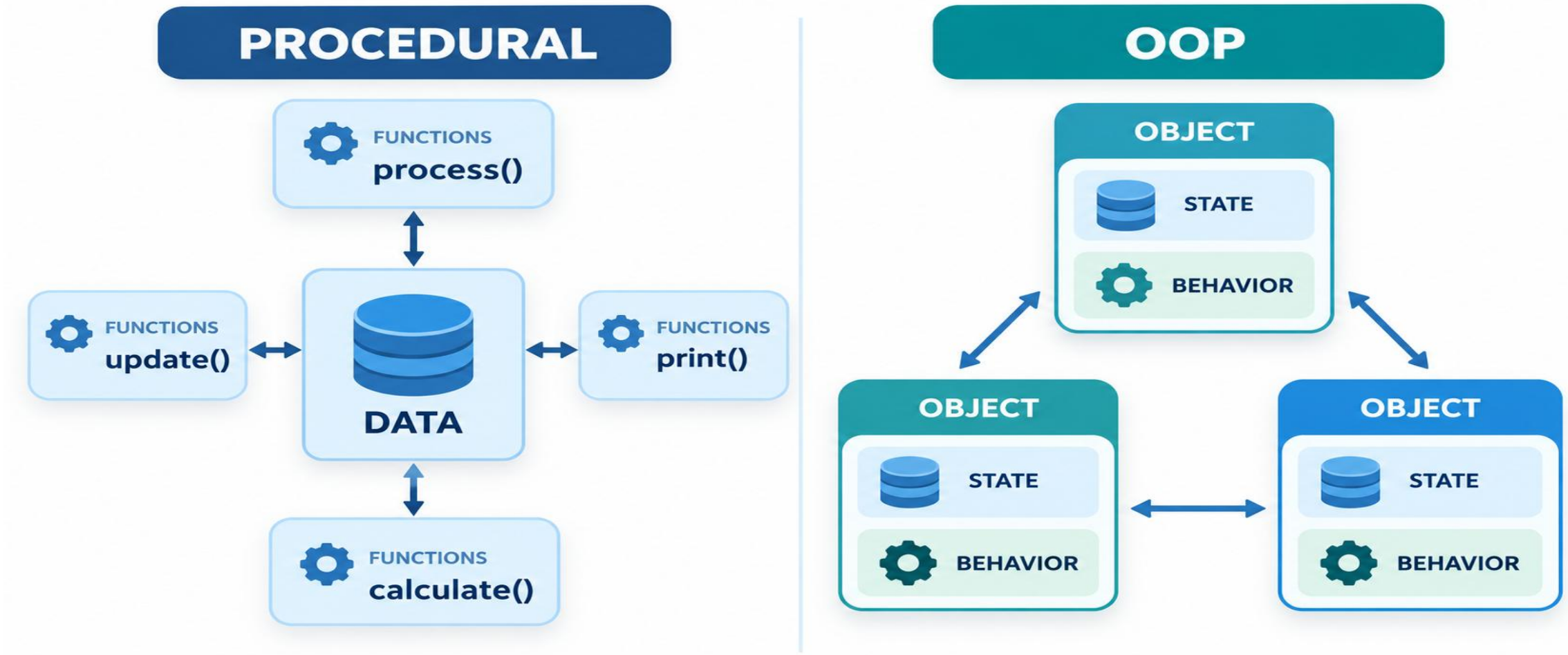
The recurring pressure: programs became larger, longer-lived, and developed by larger teams.

OOP is one response to software complexity: organize related state and operations together.

Visual model — where do state and behavior live?

VISUAL

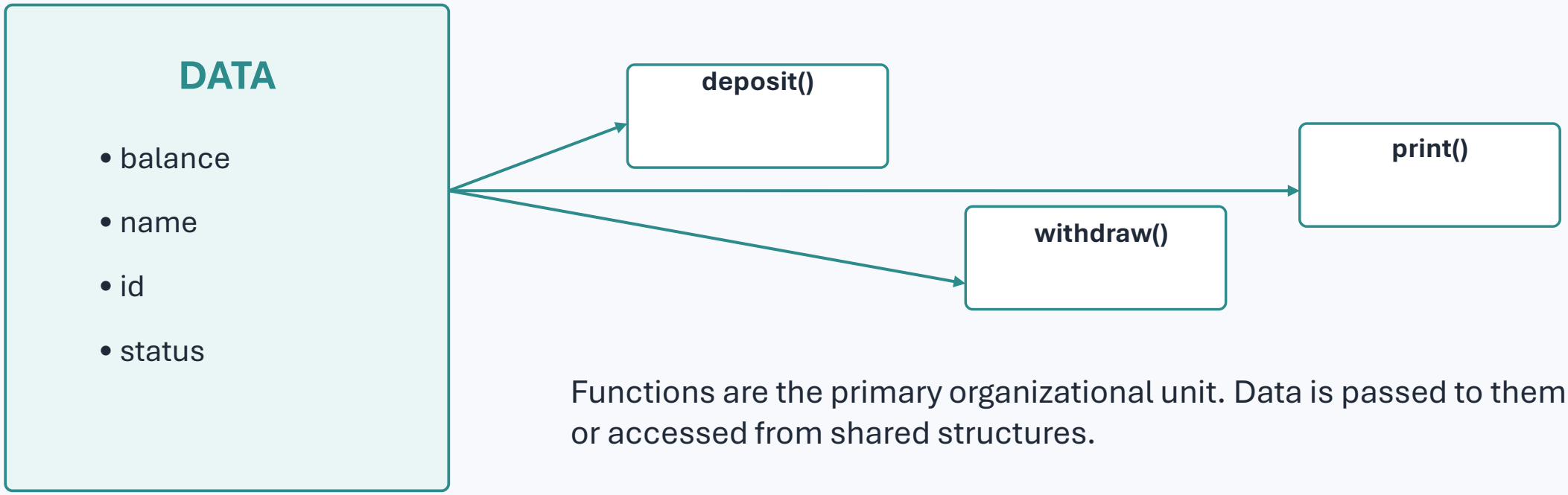
Procedural organization keeps operations separate from data; OO organization groups state and behavior inside responsible objects.



Procedural programming: the core idea

PROCEDURAL

Break a problem into a sequence of procedures that operate on data.



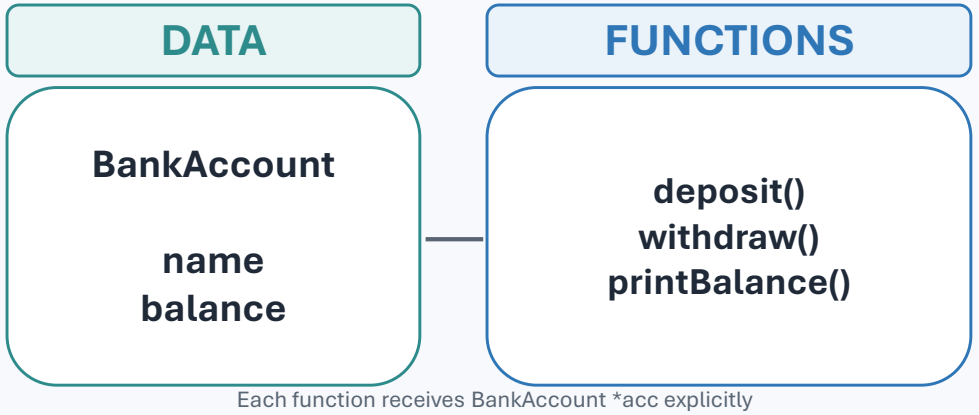
Functions are the primary organizational unit. Data is passed to them or accessed from shared structures.

C example — data and functions are separate

PROCEDURAL C

A C struct groups related data, but ordinary functions remain outside the struct and receive that data explicitly.

```c
#include <stdio.h>

typedef struct {
    char name[30];
    double balance;
} BankAccount;          // DATA

void deposit(BankAccount *acc, double amount) {
    acc->balance += amount;
}

void withdraw(BankAccount *acc, double amount) {
    if (amount <= acc->balance)
        acc->balance -= amount;
}

void printBalance(const BankAccount *acc) {
    printf("%.2f\n", acc->balance);
}

int main(void) {
    BankAccount a = {"Ali", 1000.0};
    deposit(&a, 500.0);
    withdraw(&a, 200.0);
    printBalance(&a);
    return 0;
}
```



The functions are NOT members of BankAccount.

C / procedural	Java / OOP
`deposit(&a, 500);`	`a.deposit(500);`
Give the data to a function	Ask the object to act

Running example: a tiny bank account program

EXAMPLE

Suppose we need to store a balance, deposit money, and withdraw money.

```
double balance = 1000.0;

void deposit(double amount) {
    balance = balance + amount;
}

void withdraw(double amount) {
    balance = balance - amount;
}

void printBalance() {
    System.out.println(balance);
}
```

- The state (balance) exists separately from the operations.
- Each procedure assumes it is operating on the correct data.
- This is easy to understand for one account and a very small program.

What changes when we need 10,000 accounts, different account types, and many developers?

Procedural Java example with explicit data passing

CODE

Java can be written procedurally by using static methods and data values.

```
public class BankProcedural {
    static double deposit(double balance, double amount)
{
        return balance + amount;
    }

    static double withdraw(double balance, double
amount) {
        return balance - amount;
    }

    public static void main(String[] args) {
        double balance = 1000.0;
        balance = deposit(balance, 500.0);
        balance = withdraw(balance, 200.0);
        System.out.println(balance);
    }
}
```

Output

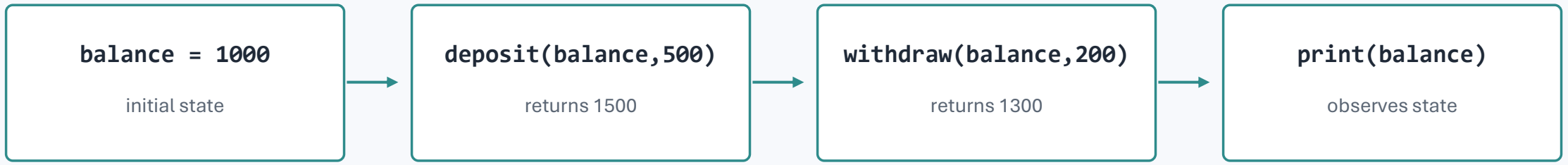
```
1300.0
```

- No object is required to express this solution.
- The approach is perfectly valid for a small problem.
- The design question is about scalability of organization, not syntax.

How a procedural solution executes

FLOW

Think in terms of a sequence of transformations on state.



The logic is explicit and linear—a major strength for small computational tasks.

What procedural programming does well

STRENGTHS

OOP is not a replacement for all procedural thinking.

Good fit

- Short scripts and utilities
- Straightforward numerical transformations
- Algorithms with a clear sequence of steps
- Small programs with limited shared state

Benefits

- Simple control flow
- Low conceptual overhead
- Direct mapping from algorithm to code
- Often easy to test at function level

Inside OOP methods, we still write procedural statements and algorithms.

Structured programming: discipline over control flow

STRUCTURED

Structured programming improved reliability by avoiding uncontrolled jumps.

Sequence

`Do A, then B, then C`

Selection

`if / else / switch`

Iteration

`for / while / do-while`

Decomposition

methods / modules

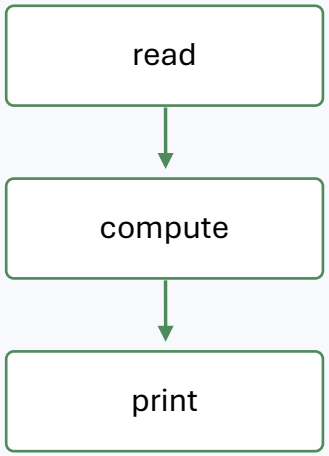
Structured programming improves HOW control flows. OOP mainly changes HOW responsibilities and data are organized.

The three structured control constructs

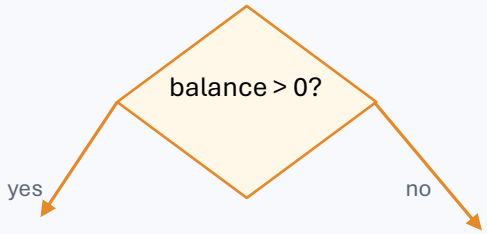
CONTROL FLOW

Most algorithmic logic can be built from these patterns.

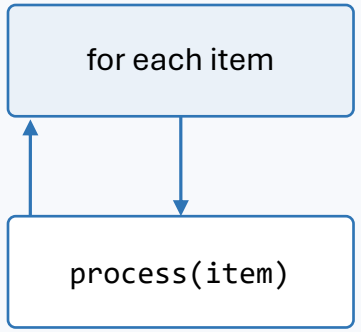
SEQUENCE



SELECTION



ITERATION



OOP does not eliminate these constructs—it gives them a home inside cohesive objects.

Structured refactoring: from “one long main” to methods

REFACTOR

First improvement: split a large algorithm into meaningful procedures.

```
public static void main(String[] args) {
    double balance = 1000;
    balance = deposit(balance, 500);
    balance = withdraw(balance, 200);
    printBalance(balance);
}

static double deposit(double b, double a) {
... }
static double withdraw(double b, double a) {
... }
static void printBalance(double b) { ... }
```

- Names communicate intent.
- Each method has one focused task.
- Logic can be tested independently.
- But the account state is still external to the operations.

Structured decomposition solves one dimension of complexity. Data organization remains another.

Procedural vs. structured: what actually changed?

COMPARE

“Structured” is usually a disciplined form of procedural programming.

Aspect	Procedural	Structured
Primary unit	Procedure / function	Procedure / function
Control flow	May include arbitrary jumps	Sequence, selection, iteration
Decomposition	Possible	Strongly encouraged
Main improvement	Direct step-by-step execution	Readability and reasoning
Data organization	Usually separate from procedures	Usually still separate from procedures

When procedural organization is enough

JUDGMENT

The simplest correct design is often the best design.

CSV converter

```
read → transform → write
```

Math utility

```
pure calculations
```

Build script

```
sequence of automation steps
```

Small algorithmic task

```
clear inputs and outputs
```

Avoid “OOP for OOP’s sake.” Use objects when they help model responsibilities, state, variation, and collaboration.

The scaling problem

MOTIVATION

A design that is easy at 200 lines can become fragile at 200,000 lines.

More data

many entities & states

More behavior

many rules & operations

More change

requirements evolve

More people

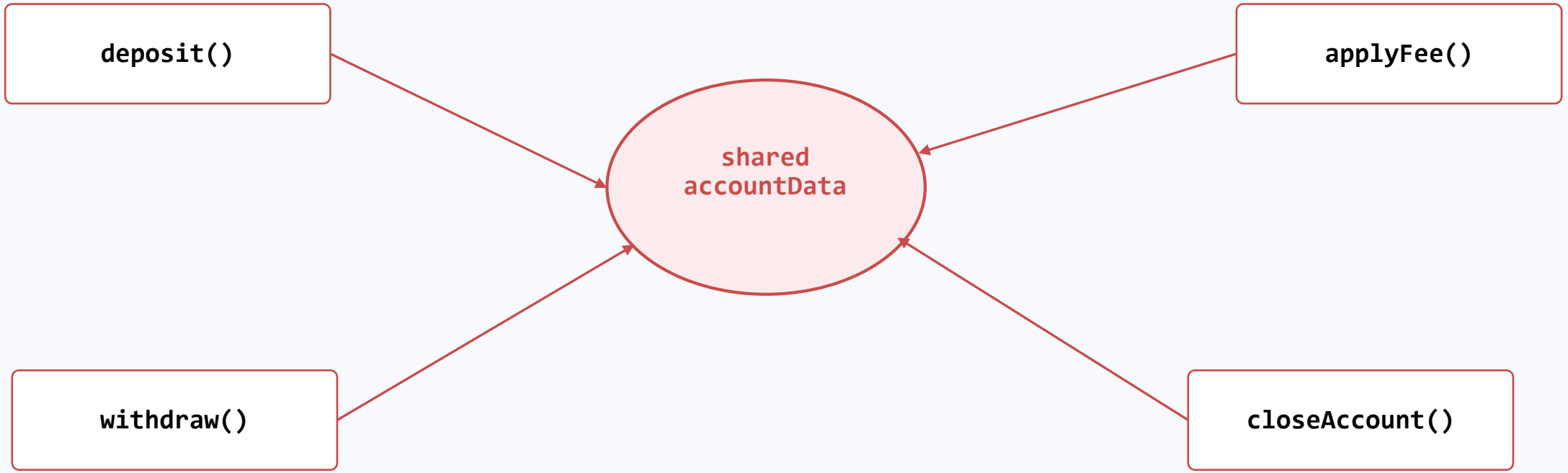
teams work in parallel

Complexity grows from interactions between parts—not just from the number of lines.

Scaling issue #1: shared mutable data

LIMITATION

When many procedures can modify the same data, hidden dependencies appear.



A change in one procedure can violate an assumption made by another procedure.

Scaling issue #2: change ripples across the program

LIMITATION

New requirements often require coordinated edits in many places.

New rule: “Savings accounts cannot go below Rs. 0.”

`withdraw()`	`transfer()`	`feeProcess()`	`importTransactions()`	`tests`
update validation	duplicate/check rule	avoid negative result	validate imported data	update expected behavior

When one concept is scattered across many functions, the cost of change increases.

Scaling issue #3: duplicated rules and weak reuse

LIMITATION

Copying logic is easy. Keeping copies consistent is hard.

```
// Module A
if (amount > 0 && amount <= balance) { ...
}

// Module B
if (amount >= 0 && amount < balance) { ...
}

// Module C
if (balance - amount >= 0) { ... }
```

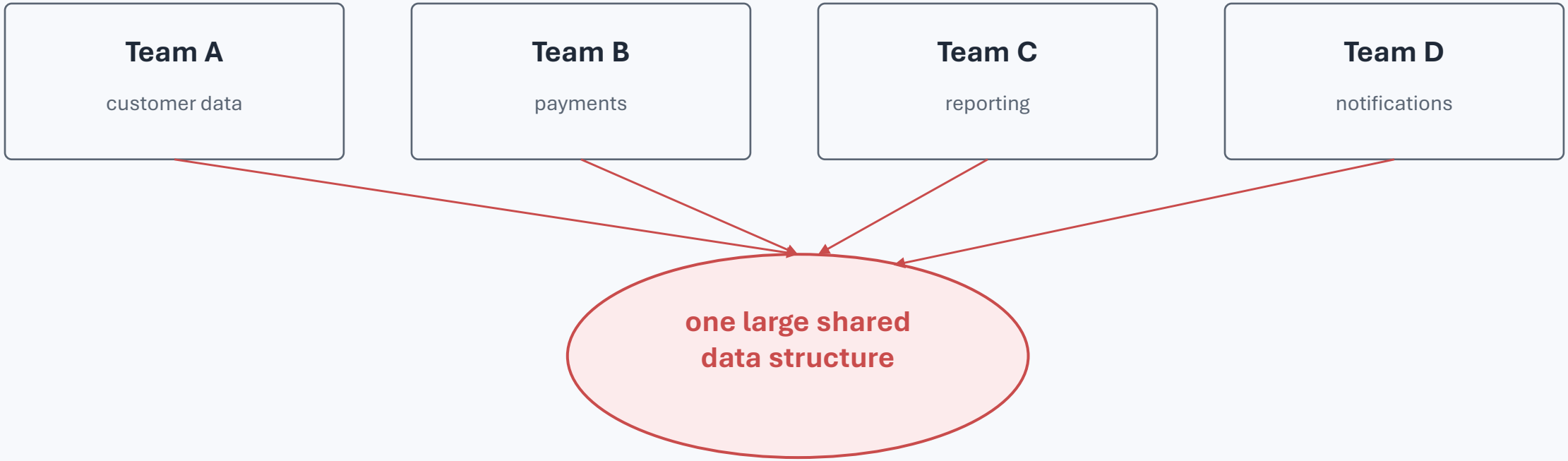
Same business rule — three slightly different versions

- Bug fixes may be applied to only one copy.
- Behavior becomes inconsistent across the system.
- Developers must remember where every related rule lives.

Scaling issue #4: team development and ownership

LIMITATION

Large systems need clear boundaries so teams can work without breaking each other’s code.



Without boundaries, “who owns this data/rule?” becomes unclear.

Case study: a student record system grows

CASE STUDY

A simple program becomes a system with many rules and actors.

Data structures

- Student name
- ID
- CGPA
- Courses
- Fee status

Procedures

- registerCourse()
- dropCourse()
- calculateGPA()
- payFee()
- printTranscript()

As rules grow, the mapping between “which data” and “which procedures” becomes harder to maintain.

OOP redesign: give each concept responsibility for its own state

TRANSITION

Move related data and behavior into a cohesive abstraction.

Student

STATE

`id, name, cgpa`

BEHAVIOR

`registerCourse()`
`calculateGPA()`

Course

STATE

`code, title, credits`

BEHAVIOR

`enroll()`
`withdraw()`

Account

STATE

`balance, status`

BEHAVIOR

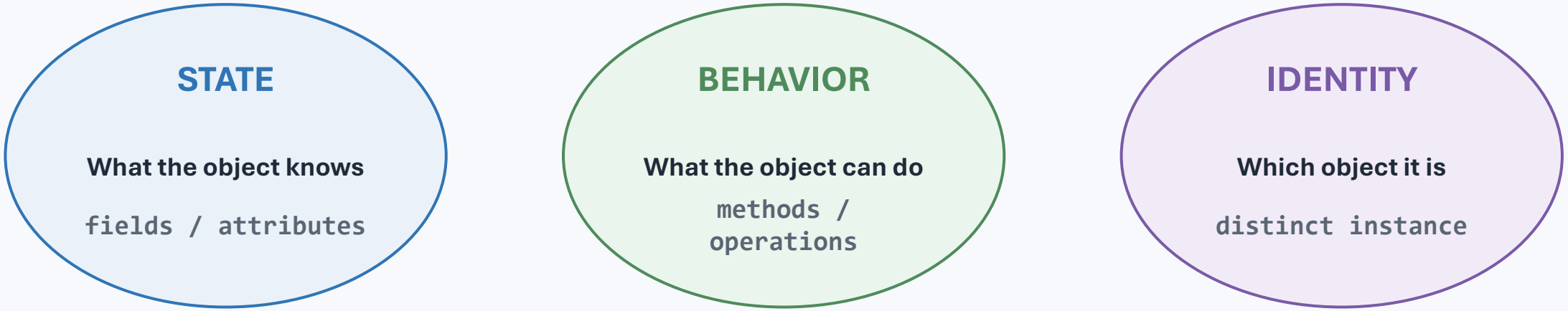
`pay()`
`refund()`

OOP asks: What object should be responsible for this state and this behavior?

Object-oriented programming: the central idea

OOP

Model software as collaborating objects with state, behavior, and identity.



An object should protect its state and expose meaningful operations rather than arbitrary data manipulation.

Anatomy of an object

OBJECTS

An object is more than a record of values.

Identity

State

bankAccount #A102

STATE

```
balance = 1300
status = ACTIVE
```

BEHAVIOR

```
deposit(amount)
withdraw(amount)
getBalance()
```

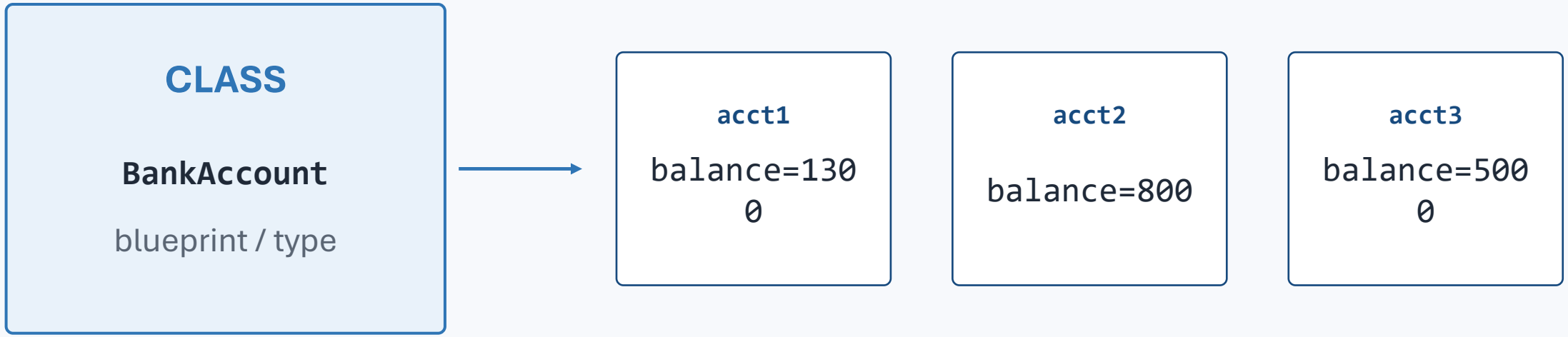
Behavior

Real-world view: one account owns its state and permitted actions.

Class vs. object — a first preview

PREVIEW

A class describes a kind of object; objects are concrete instances.



One class → many independent objects

The same bank example, now object-oriented

JAVA PREVIEW

The account owns its balance and the operations that may change it.

```
class BankAccount {
    private double balance;

    BankAccount(double openingBalance) {
        balance = openingBalance;
    }

    void deposit(double amount) {
        balance += amount;
    }

    void withdraw(double amount) {
        if (amount <= balance) balance -= amount;
    }

    double getBalance() { return balance; }
}
```

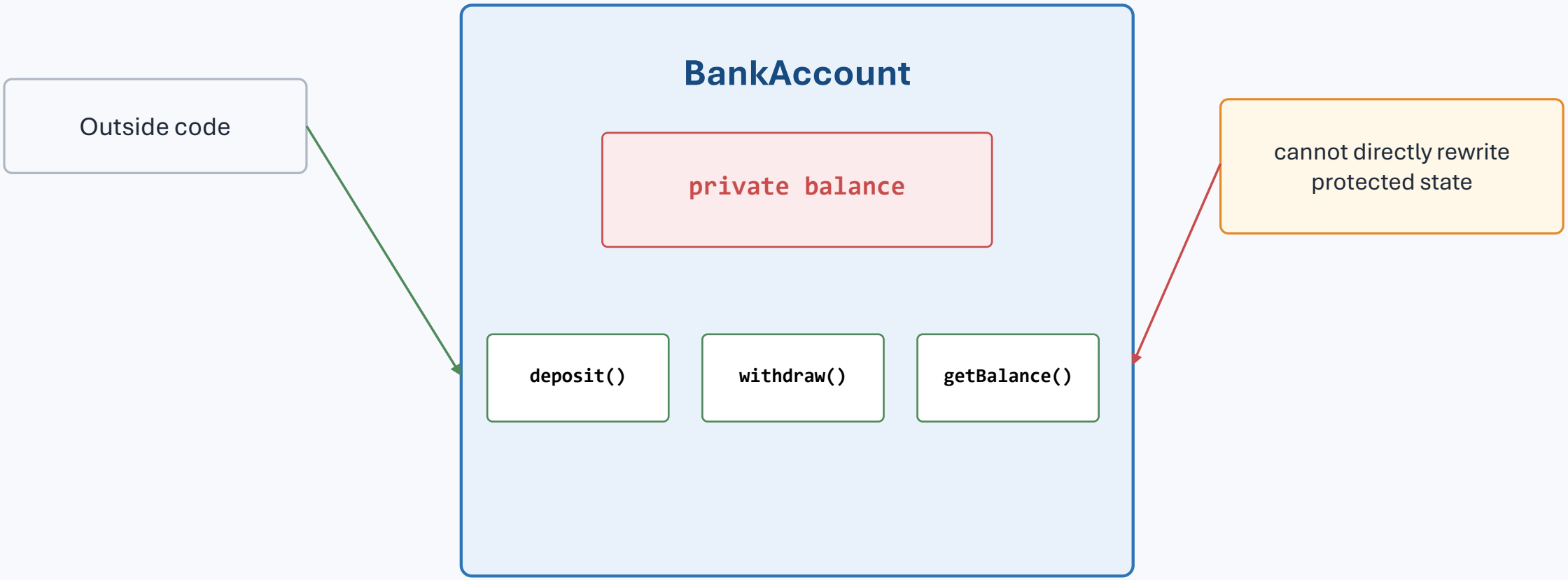
```
BankAccount a = new BankAccount(1000);
a.deposit(500);
a.withdraw(200);
System.out.println(a.getBalance());
```

- State and behavior live together.
- Validation can be centralized inside the object.
- Client code asks the object to perform meaningful operations.

Encapsulation: a preview of the protection boundary

OOP

The object becomes the controlled gateway to its own state.

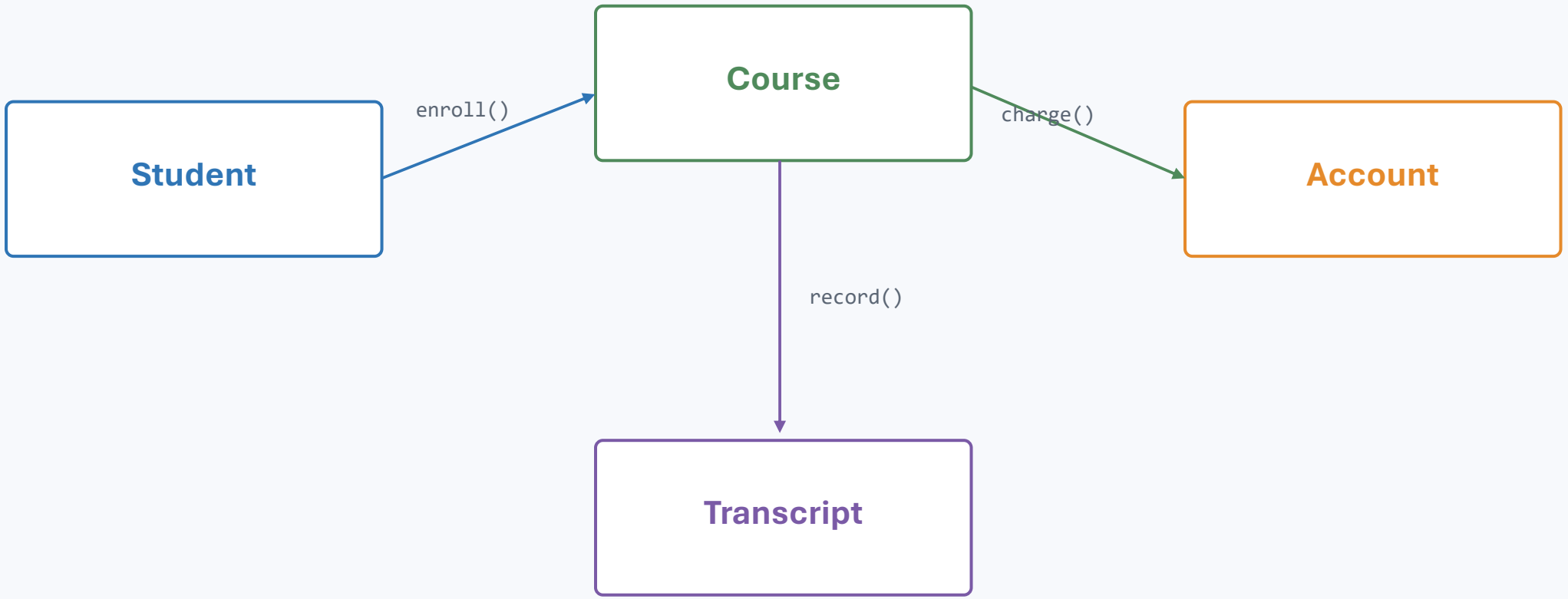


Detailed access modifiers and encapsulation are covered later; here we focus on the motivation.

Objects collaborate by sending requests to each other

COLLABORATION

A system becomes a network of responsibilities.

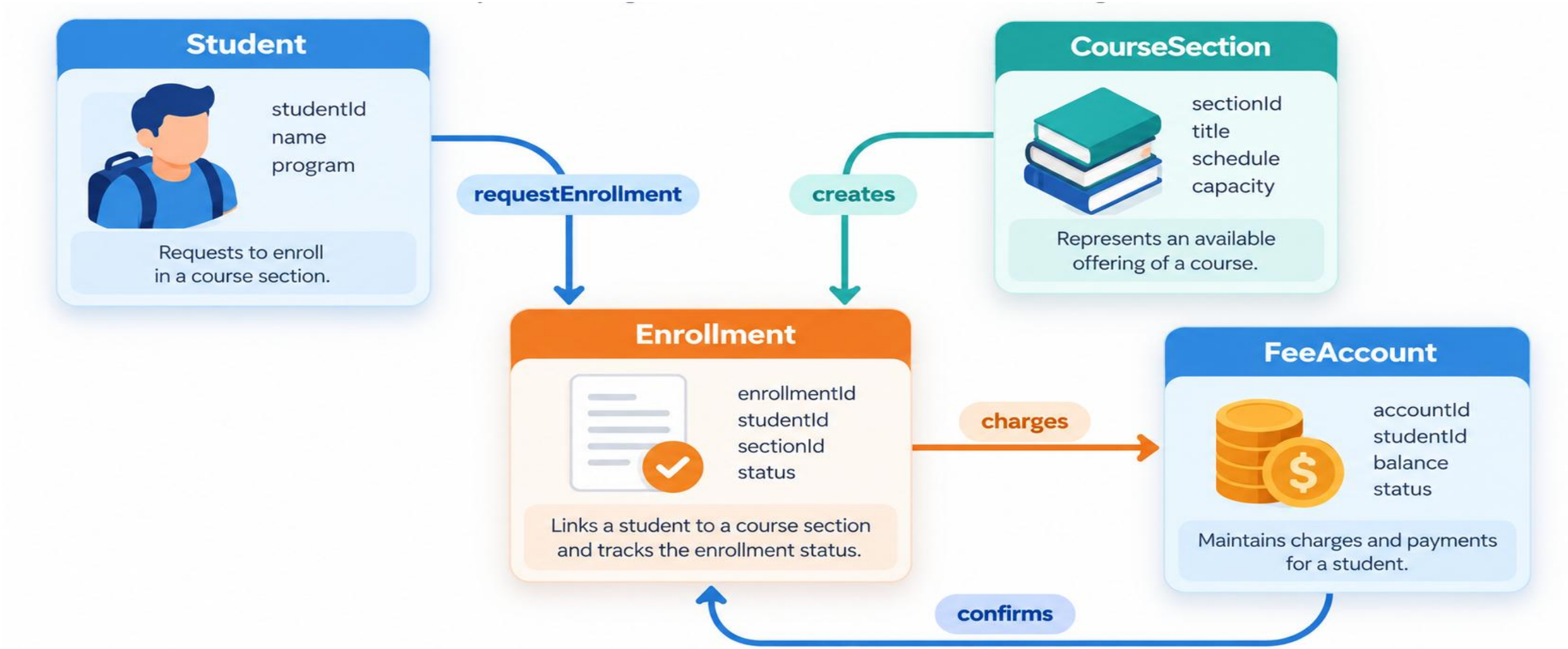


Good OO design is not “put everything in classes.” It is assigning responsibilities to the right objects.

Visual model — object collaboration

COLLABORATION

A single feature often crosses several responsible objects; OOP is about collaboration, not isolated classes.



From a problem statement to candidate objects

THINKING

A useful first instinct: look for concepts that own information and responsibilities.

Problem statement

“A library member borrows books. The system records the due date, checks whether a book is available, and calculates a fine for late returns.”

Member

Book

Loan

Fine

Candidate objects

Later lectures formalize this with object identification, CRC thinking, and UML.

Paradigm comparison at a glance

SUMMARY

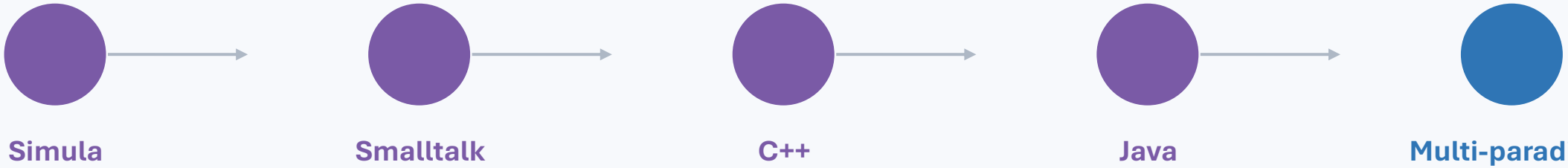
Focus on the dominant unit of organization.

Aspect	Procedural	Structured	Object-Oriented
Organizing unit	procedures/functions	procedures + structured flow	objects/classes
Data & behavior	mostly separate	mostly separate	bundled by responsibility
Best mental model	steps in an algorithm	well-structured steps	collaborating entities
Change localization	depends on modularity	better than unstructured code	often stronger around domain concepts
Typical strength	simplicity	clarity of control flow	modularity + extensibility
Typical risk	shared data / scattering	still data-centric	over-engineering / poor class design

Brief history of object-oriented ideas

HISTORY

The idea evolved over decades and influenced mainstream software development.



classes & objects for simulation

object/message-centered environment

OOP added to systems programming

portable, class-based mainstream OOP

OOP coexists with functional & other styles

History matters because OOP emerged as a response to modeling and software-organization challenges—not as a syntax feature.

Why OOP became attractive for large software

WHY OOP

OOP gives developers a vocabulary for boundaries, responsibilities, reuse, and variation.

Localize change

rules live near the state they govern

Reuse abstractions

share stable behavior across the system

Model domains

classes mirror important concepts

Support teams

clear interfaces reduce accidental coupling

Enable extensibility

new variants can fit common abstractions

Work with frameworks

APIs often expect object contracts

These benefits are design possibilities, not automatic guarantees. Poorly designed OOP can still be complex.

Think–pair–share: choose the organizing style

ACTIVITY

For each problem, argue whether procedural decomposition or an OO model is more natural—and why.

A Convert 5,000 CSV files to JSON

B Build a university registration system

C Compute factorials and prime numbers

D Develop an online shopping platform

Defend your choice using: state, number of entities, change frequency, collaboration, and complexity.

Lecture recap & exit ticket

RECAP

The key transition: from functions operating on data → objects that own data and behavior.

Remember

- Procedural: organize around procedures
- Structured: disciplined control flow
- OOP: organize around responsible objects
- OOP helps localize state and behavior

Exit ticket

- What problem does OOP try to solve?
- Give one case where procedural design is preferable.
- In your own words: what is an object?
- Why can shared mutable data become risky?

Next lecture: Advantages of OOP & the Modern Java Landscape — JDK, JVM, Java SE, compilation and execution pipeline.